

**Name : Kadava Yogi**

**Reg no : 192372053**

**Dept : CSE(AI)**

**1. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with a and end with a**

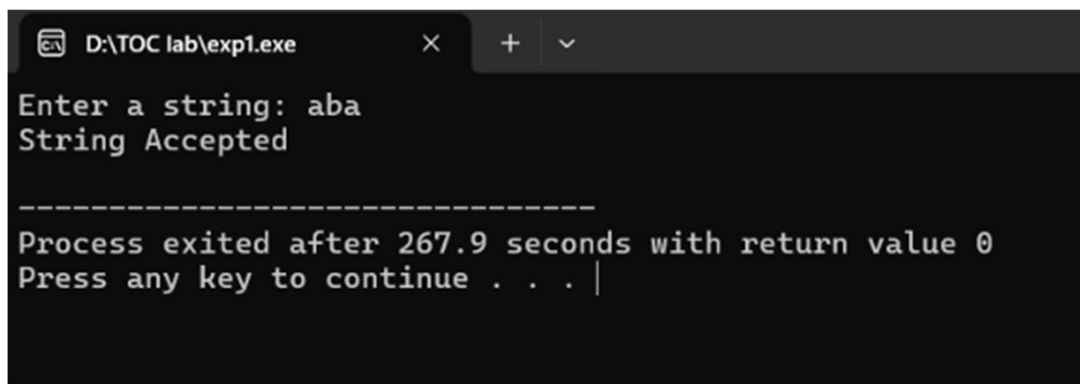
**CODE:**

```
#include <stdio.h>

#include <string.h>

int main() {
    char str[100];
    int n;
    printf("Enter a string: ");
    scanf("%s", str);
    n = strlen(str);
    if (str[0] == 'a' && str[n - 1] == 'a')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");
    return 0;
}
```

**OUTPUT:**



```
D:\TOC lab\exp1.exe
Enter a string: aba
String Accepted

-----
Process exited after 267.9 seconds with return value 0
Press any key to continue . . . |
```

**2. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given**

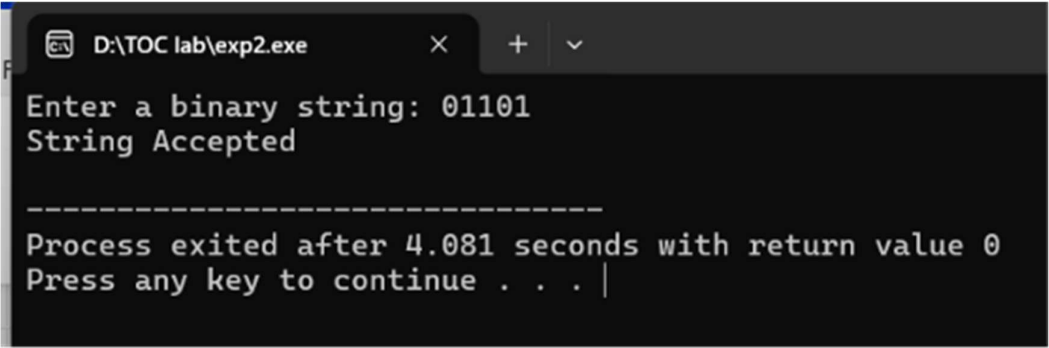
**language representing strings that start with 0 and end with 1**

**CODE:**

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[100];
    int n;
    printf("Enter a binary string: ");
    scanf("%s", str);
    n = strlen(str);
    if (str[0] == '0' && str[n - 1] == '1')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");
    return 0;
}
```

**OUTPUT:**



```
D:\TOC lab\exp2.exe
Enter a binary string: 01101
String Accepted

-----
Process exited after 4.081 seconds with return value 0
Press any key to continue . . . |
```

**3. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)**

$S \rightarrow 0A1$        $A \rightarrow 0A \mid 1A \mid \epsilon$

**CODE:**

```

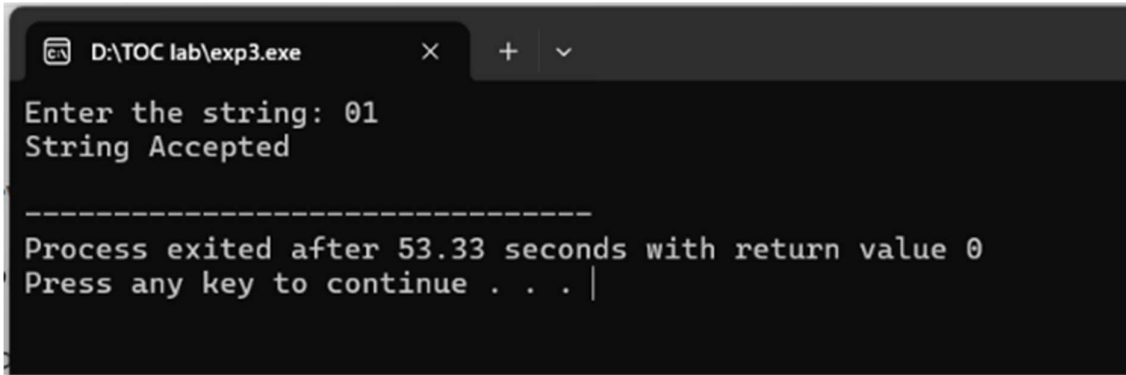
#include <stdio.h>
#include <string.h>
int main()
{
    char str[100];
    int n, i, valid = 1;
    printf("Enter the string: ");
    scanf("%s", str);
    n = strlen(str);
    // Minimum length is 2 (01)
    if (n < 2)
    {
        printf("String Rejected\n");
        return 0;
    }
    // Check whether all characters are 0 or 1
    for (i = 0; i < n; i++)
    {
        if (str[i] != '0' && str[i] != '1')
        {
            valid = 0;
            break;
        }
    }

    if (valid && str[0] == '0' && str[n - 1] == '1')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");
}

```

```
    return 0;
}
```

#### OUTPUT:



```
D:\TOC lab\exp3.exe
Enter the string: 01
String Accepted

-----
Process exited after 53.33 seconds with return value 0
Press any key to continue . . . |
```

4. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)

$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$

#### CODE:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[100];
    int i, n, flag = 1;
    printf("Enter a binary string: ");
    scanf("%s", str);
    n = strlen(str);
    // Check whether the string contains only 0 and 1
    for(i = 0; i < n; i++)
    {
        if(str[i] != '0' && str[i] != '1')
        {
            printf("String Rejected\n");
```

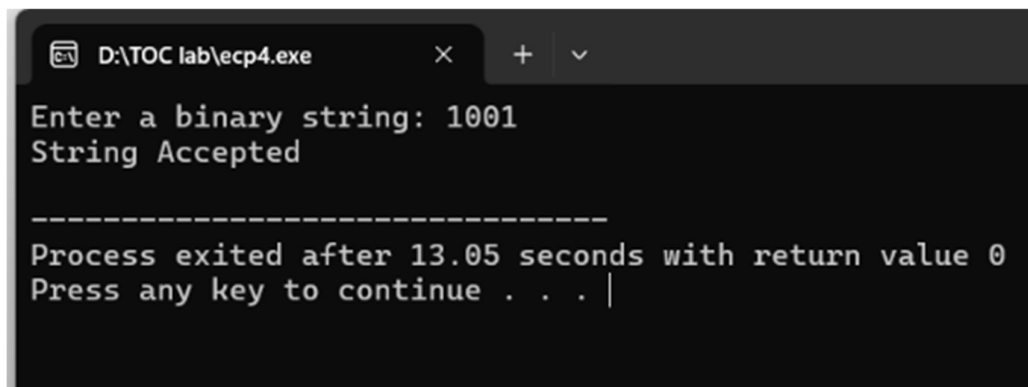
```

        return 0;
    }
}
// Check palindrome
for(i = 0; i < n / 2; i++)
{
    if(str[i] != str[n - i - 1])
    {
        flag = 0;
        break;
    }
}
if(flag)
    printf("String Accepted\n");
else
    printf("String Rejected\n");

return 0;
}

```

#### OUTPUT:



```

D:\TOC lab\ecp4.exe
Enter a binary string: 1001
String Accepted

-----
Process exited after 13.05 seconds with return value 0
Press any key to continue . . . |

```

**5. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)**

$S \rightarrow 0S0 \mid A A \rightarrow 1A \mid \epsilon$

**CODE:**

```
#include <stdio.h>
#include <string.h>

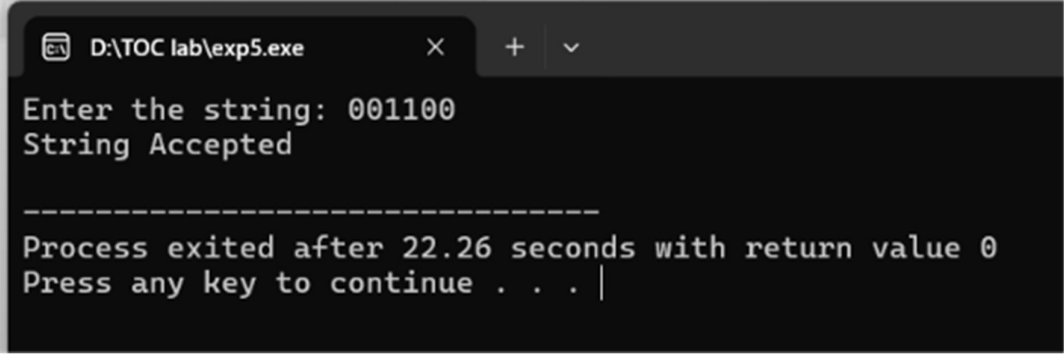
int main()
{
    char str[100];
    int i = 0, j, n;
    int left0 = 0, right0 = 0;
    printf("Enter the string: ");
    scanf("%s", str);
    n = strlen(str);
    j = n - 1;
    // Count leading 0's
    while (i < n && str[i] == '0')
    {
        left0++;
        i++;
    }
    // Count trailing 0's
    while (j >= 0 && str[j] == '0')
    {
        right0++;
        j--;
    }
    // Number of leading and trailing 0's must be equal
    if (left0 != right0)
    {
        printf("String Rejected\n");
    }
}
```

```

    return 0;
}
// Middle part should contain only 1's
for (i = left0; i <= j; i++)
{
    if (str[i] != '1')
    {
        printf("String Rejected\n");
        return 0;
    }
}
printf("String Accepted\n");
return 0;
}

```

#### OUTPUT:



```

D:\TOC lab\exp5.exe
Enter the string: 001100
String Accepted

-----
Process exited after 22.26 seconds with return value 0
Press any key to continue . . . |

```

**6. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)**

$S \rightarrow 0S1 \mid \epsilon$

#### CODE:

```

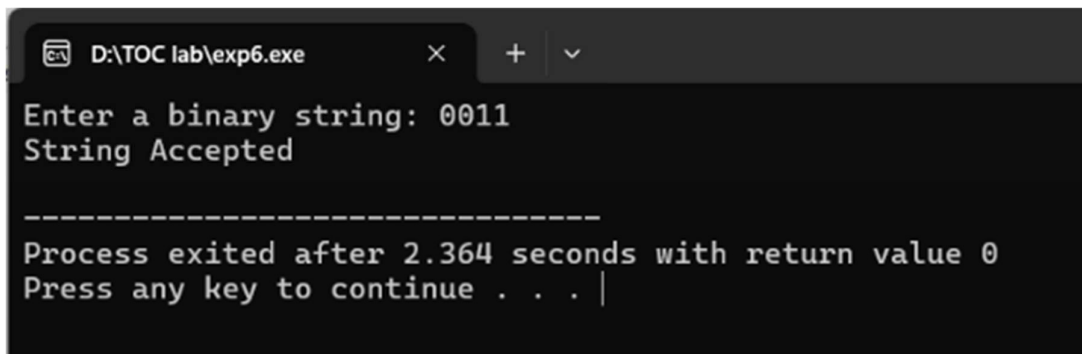
#include <stdio.h>
#include <string.h>

int main()
{

```

```
char str[100];  
int i = 0, zeros = 0, ones = 0, n;  
printf("Enter a binary string: ");  
scanf("%s", str);  
n = strlen(str);  
// Count leading 0's  
while (i < n && str[i] == '0')  
{  
    zeros++;  
    i++;  
}  
// Count following 1's  
while (i < n && str[i] == '1')  
{  
    ones++;  
    i++;  
}  
// If extra characters remain or counts are unequal  
if (i != n || zeros != ones)  
    printf("String Rejected\n");  
else  
    printf("String Accepted\n");  
return 0;  
}
```

## OUTPUT:



```
D:\TOC lab\exp6.exe
Enter a binary string: 0011
String Accepted

-----
Process exited after 2.364 seconds with return value 0
Press any key to continue . . . |
```

**7. Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)**

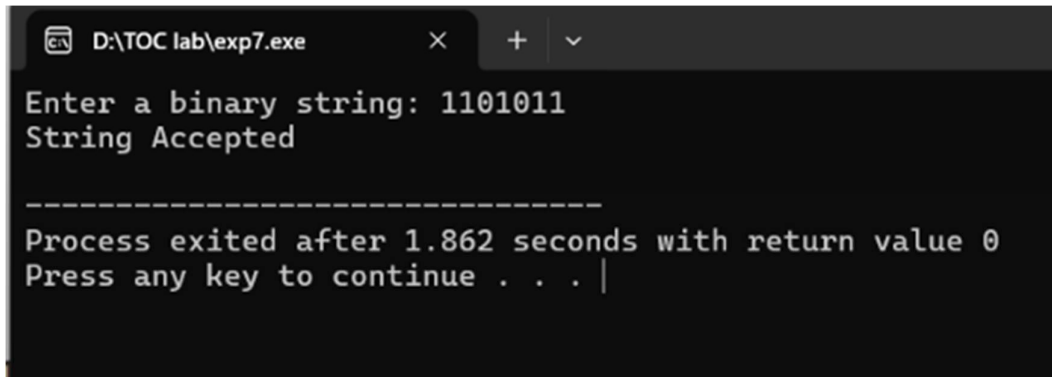
$S \rightarrow A101A, A \rightarrow 0A \mid 1A \mid \epsilon$

### CODE:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[100];
    printf("Enter a binary string: ");
    scanf("%s", str);
    if (strstr(str, "101") != NULL)
        printf("String Accepted\n");
    else
        printf("String Rejected\n");
    return 0;
}
```

## OUTPUT:



```
D:\TOC lab\exp7.exe
Enter a binary string: 1101011
String Accepted

-----
Process exited after 1.862 seconds with return value 0
Press any key to continue . . . |
```

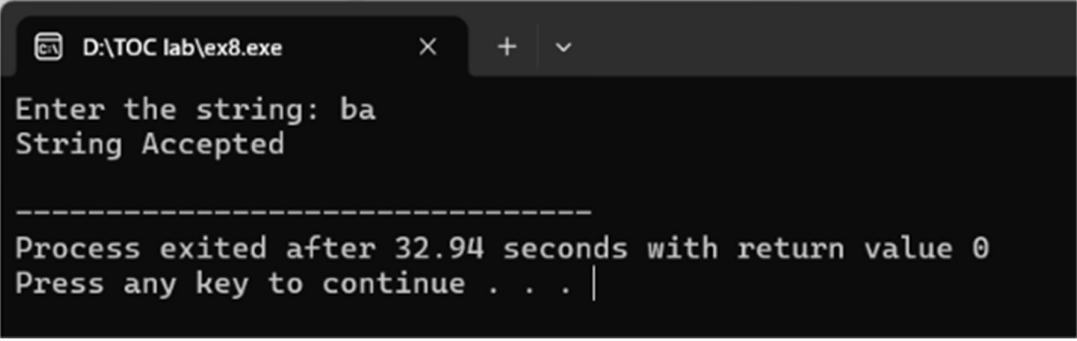
**8. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with b and end with a**

### CODE:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int len;
    printf("Enter the string: ");
    scanf("%s", str);
    len = strlen(str);
    if (len > 0 && str[0] == 'b' && str[len - 1] == 'a')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");
    return 0;
}
```

```
}
```



```
D:\TOC lab\ex8.exe
Enter the string: ba
String Accepted

-----
Process exited after 32.94 seconds with return value 0
Press any key to continue . . . |
```

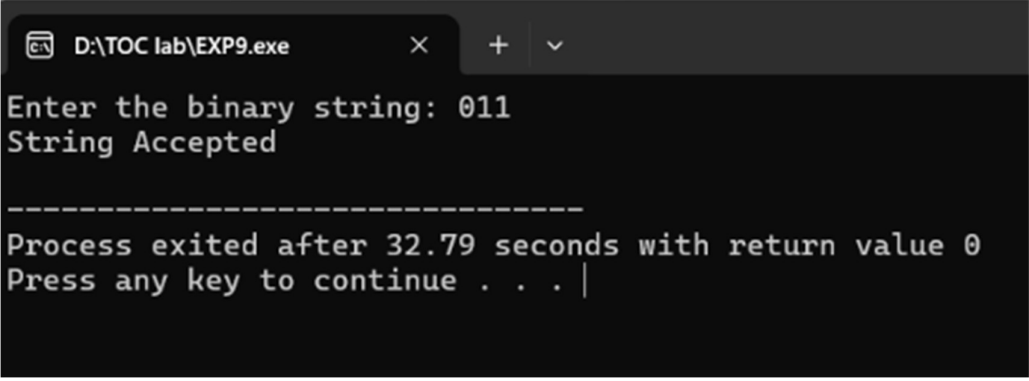
**9. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with 0 and end with 1**

**CODE:**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int len;
    printf("Enter the binary string: ");
    scanf("%s", str);
    len = strlen(str);
    if (len > 0 && str[0] == '0' && str[len - 1] == '1')
        printf("String Accepted\n");
    else
        printf("String Rejected\n");
    return 0;
}
```

```
}
```



```
D:\TOC lab\EXP9.exe
Enter the binary string: 011
String Accepted

-----
Process exited after 32.79 seconds with return value 0
Press any key to continue . . . |
```

**10. Write a C program to find  $\epsilon$ -closure for all the states in a Non-Deterministic Finite Automata (NFA) with  $\epsilon$ -moves.**

**CODE:**

```
#include <stdio.h>

int n;
int e[10][10];
int visited[10];

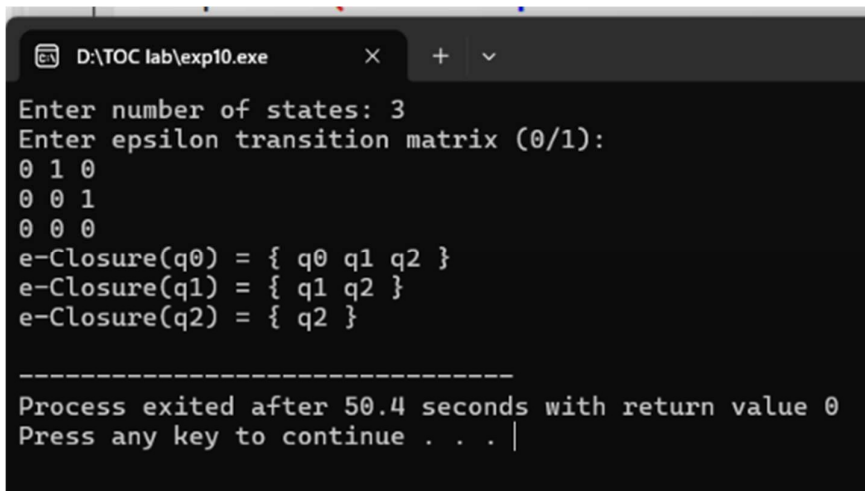
void epsilonClosure(int state)
{
    int i;
    visited[state] = 1;
    printf("q%d ", state);
    for(i = 0; i < n; i++)
    {
        if(e[state][i] && !visited[i])
            epsilonClosure(i);
    }
}

int main()
{
    int i, j;
    printf("Enter number of states: ");
```

```

scanf("%d", &n);
printf("Enter epsilon transition matrix (0/1):\n");
for(i = 0; i < n; i++)
    for(j = 0; j < n; j++)
        scanf("%d", &e[i][j]);
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
        visited[j] = 0;
    printf("ε-Closure(q%d) = { ", i);
    epsilonClosure(i);
    printf("}\n");
}
return 0;
}

```



```

D:\TOC lab\exp10.exe
Enter number of states: 3
Enter epsilon transition matrix (0/1):
0 1 0
0 0 1
0 0 0
e-Closure(q0) = { q0 q1 q2 }
e-Closure(q1) = { q1 q2 }
e-Closure(q2) = { q2 }

-----
Process exited after 50.4 seconds with return value 0
Press any key to continue . . . |

```